

Wydział Informatyki Politechniki Białostockiej

Systemy wbudowane

Tytuł: Sumatory, ALU i multiplikatory

Wykonujący: Łukasz Modzelewski, Mariusz Rostkowski

PS 1

Zespół 4

Praca laboratoryjna 8

Studia dzienne

Informatyka

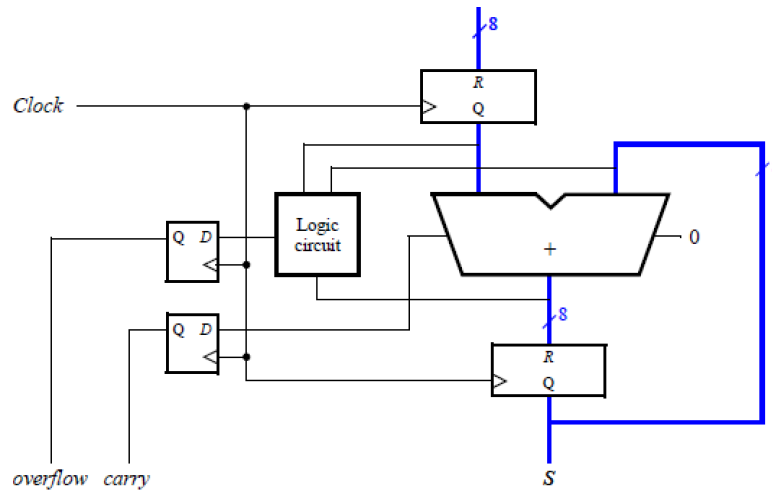
Semestr VI

Prowadzący: prof. dr hab. inż. Valery Salauyou

Data: 28.04.2022

Zadanie 1

Utwórz projekt accum_N_bits akumulatora N-bitowego, którego struktura jest pokazana na Rys. 1.

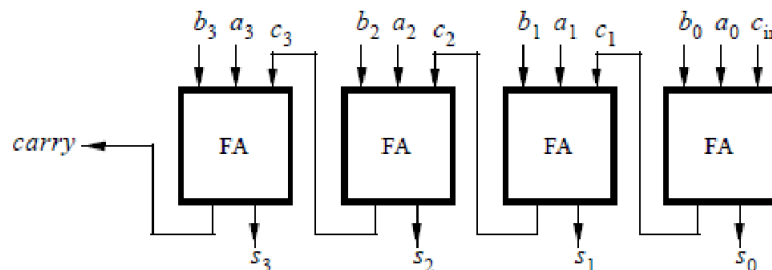


Ryc.1. Schemat ośmiobitowego akumulatora

z następującymi wyprowadzeniami:

Wyprowadzenia	Piny
A	SW7-0, HEX3-2
clk	KEY1
aclr	KEY0
S	LEDR7-0, HEX1-0
carry	LEDR8
overflow	LEDR9

Aby zaimplementować moduł sumujący, użyj układu sumatora z przeniesieniami (wzorując się na rys. 2).



Ryc. 2. Czterobitowy sumator z przeniesieniem

W drugiej wersji projektu użyj operatora przypisania

```
wire [N-1:0] A, B, sum;
assign {carry, sum} = A + B;
```

Porównaj dwa projekty pod kątem kosztów realizacji i prędkości, a także wyniki ich syntezy w **RTL Viewer** i w **Technology Map Viewer**.

Kod w Verilog

Moduł N-bitowego rejestru

```
module register_N_bits
#(parameter N = 8)(
  input [N-1:0] D,
  input clock, areset,
  output reg [N-1:0] Q);

  initial Q = 0;
  always @ (posedge clock, posedge areset)
    if (areset) Q <= 0;
    else Q <= D;

endmodule
```

Moduł N-bitowego akumulatora

Tabela prawdy sygnału overflow

S[N-1] – najbardziej znaczący bit aktualnej sumy akumulatora

A[N-1] – najbardziej znaczący bit liczby na wejściu akumulatora

S_plus_A[N-1] – najbardziej znaczący bit liczby na wyjściu sumatora

S[N-1]	A[N-1]	S_plus_A[N-1]	overflow
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

Wyznaczamy sygnał overflow metodą tabel Karnaugh.

S[N-1] A[N-1] \ S_plus_A[N-1]	0	1
00	0	1
01	0	0
11	1	0
10	0	0

overflow

```
overflow = (S[N-1] & A[N-1] & ~S_plus_A[N-1]) |
(~S[N-1] & ~A[N-1] & S_plus_A[N-1])
```

Wersja akumulatora z sumatorem z przeniesieniem szeregowym

```
module accum_N_bits
#(parameter N = 8)(
input [N-1:0] A,
input clk, aclr,
output [N-1:0] S,
output reg carry,
output reg overflow);

wire [N-1:0] s_plus_A;
wire ad_cout;
ripple_carry_adder #(N) ad(S, A, 1'b0, s_plus_A, ad_cout);
register_N_bits #(N) r(s_plus_A, clk, aclr, S);

wire of = (S[N-1] & A[N-1] & (~s_plus_A[N-1])) | ((~S[N-1]) & (~A[N-1]) & s_plus_A[N-1]);
always @ (posedge clk, posedge aclr)
if (aclr) begin
carry <= 0;
overflow <= 0;
end
else begin
carry <= ad_cout;
overflow <= of;
end
end

endmodule
```

Moduł N-bitowego sumatora z przeniesieniem szeregowym (Ripple-Carry Adder)

```
module ripple_carry_adder
#(parameter N = 4)(
input [N-1:0] A, B,
input cin,
output [N-1:0] S,
output cout);

wire [N-2:0] c;
generate
genvar i;
for (i = 0; i < N; i = i + 1)
begin: ad
case (i)
0: full_adder fa(A[i], B[i], cin, S[i], c[i]);
N-1: full_adder fa(A[i], B[i], c[i-1], S[i], cout);
default: full_adder fa(A[i], B[i], c[i-1], S[i], c[i]);
endcase
end
endgenerate

endmodule
```

Moduł 1-bitowego pełnego sumatora

```
module full_adder(
input a, b, cin,
output s, cout);

assign s = cin ^ (a ^ b);
assign cout = a & b | (a ^ b) & cin;

endmodule
```

Wersja akumulatora z operatorem przypisania ciągłego

```
module accum_N_bits_assign
#(parameter N = 8)(
input [N-1:0] A,
input clk, aclr,
output [N-1:0] S,
output reg carry,
output reg overflow);

wire [N-1:0] S_plus_A;
wire ad_cout;
assign {ad_cout, S_plus_A} = S + A;
register_N_bits #(N) r(S_plus_A, clk, aclr, S);

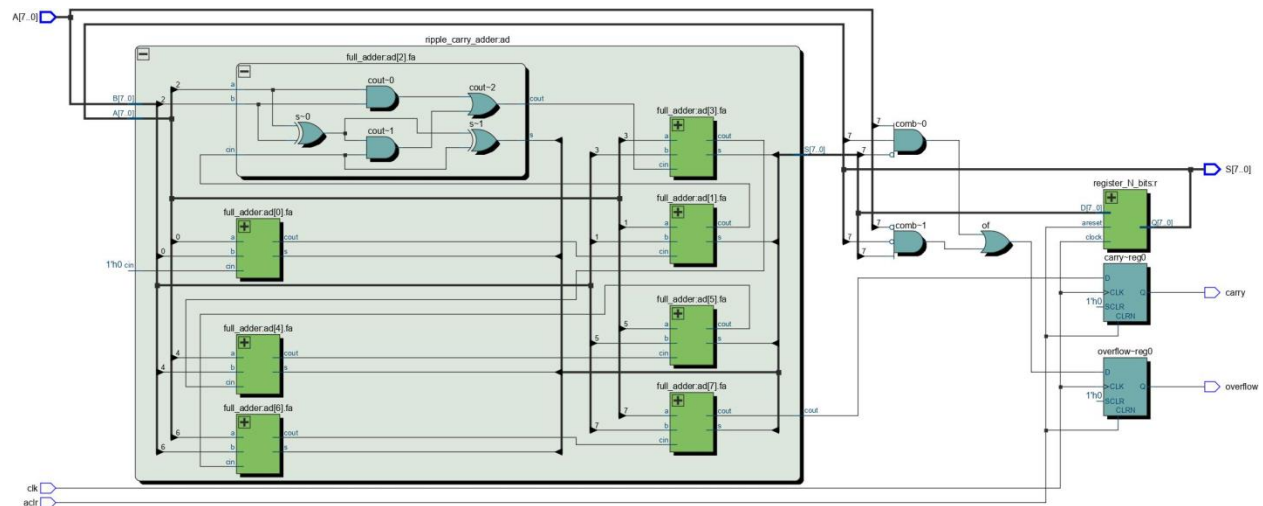
wire of = (S[N-1] & A[N-1] & (~S_plus_A[N-1])) | ((~S[N-1]) & (~A[N-1]) & S_plus_A[N-1]);
always @ (posedge clk, posedge aclr)
if (aclr) begin
carry <= 0;
overflow <= 0;
end
else begin
carry <= ad_cout;
overflow <= of;
end
end

endmodule
```

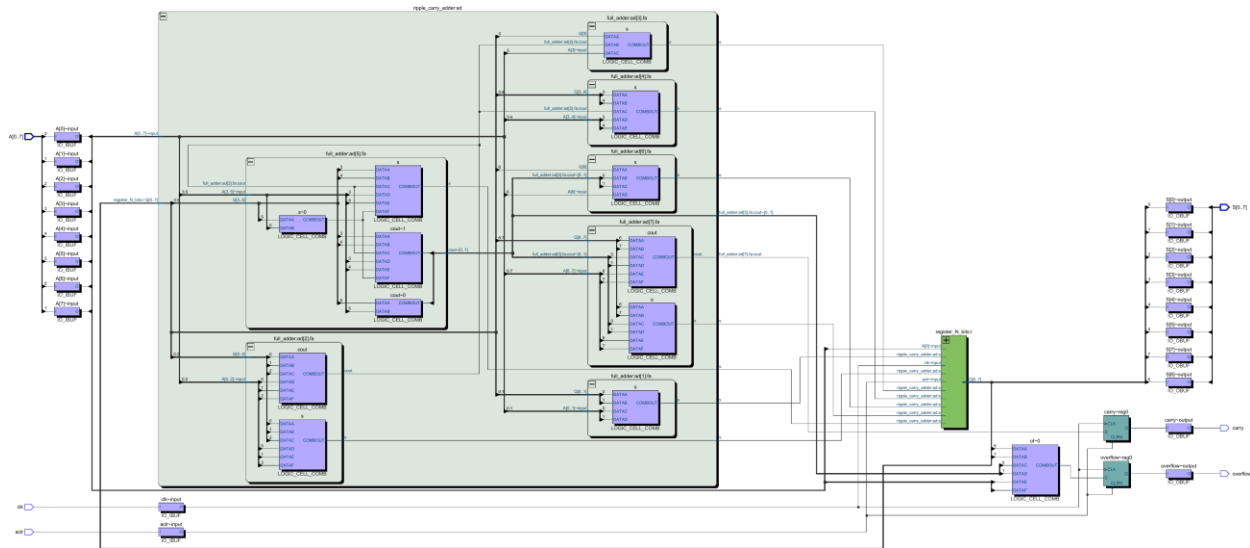
Wyniki syntezy

Akumulator z sumatorem z przeniesieniem szeregowym

RTL Viewer



Technology Map Viewer



Koszt zasobów sprzętowych

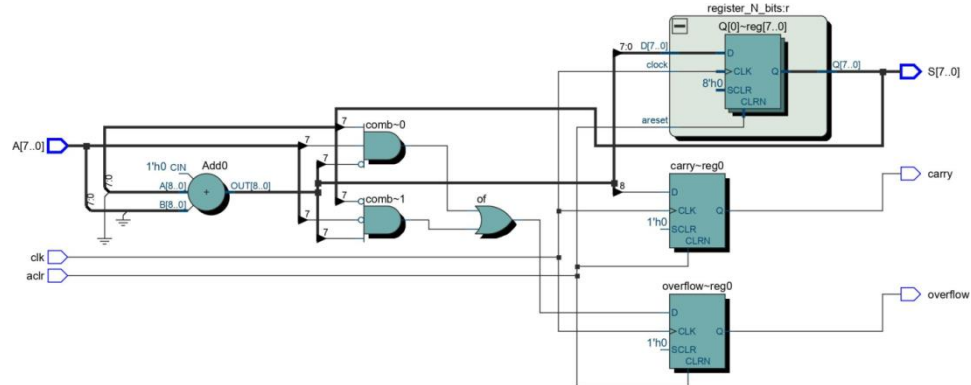
Flow Status	Successful - Tue May 03 02:20:27 2022
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	accum_N_bits
Top-level Entity Name	accum_N_bits
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	11 / 32,070 (< 1 %)
Total registers	10
Total pins	20 / 457 (4 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Maksymalna częstotliwość zegara

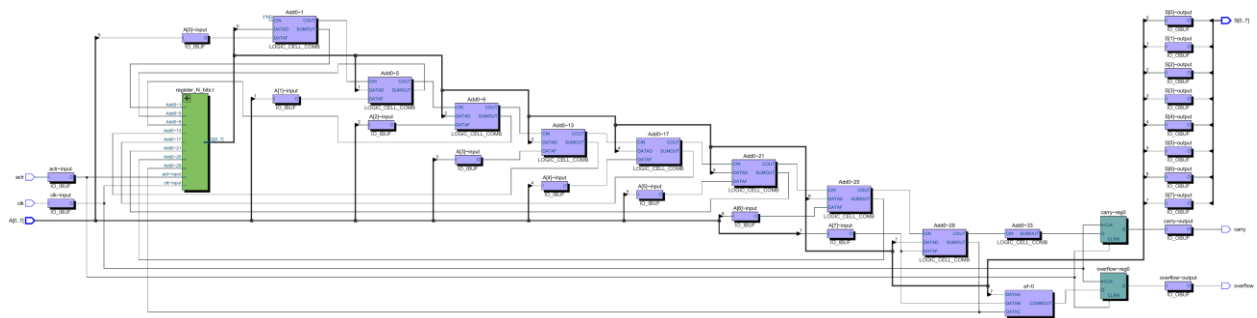
	Fmax	Restricted Fmax	Clock Name
1	547.95 MHz	547.95 MHz	clk

Akumulator z operatorem przypisania ciągłego

RTL Viewer



Technology Map Viewer



Koszt zasobów sprzętowych

Flow Status	Successful - Tue May 03 02:20:27 2022
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	accum_N_bits_assign
Top-level Entity Name	accum_N_bits_assign
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	6 / 32,070 { < 1 % }
Total registers	10
Total pins	20 / 457 { 4 % }
Total virtual pins	0
Total block memory bits	0 / 4,065,280 { 0 % }
Total DSP Blocks	0 / 87 { 0 % }
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 { 0 % }
Total DLLs	0 / 4 { 0 % }

Maksymalna częstotliwość zegara

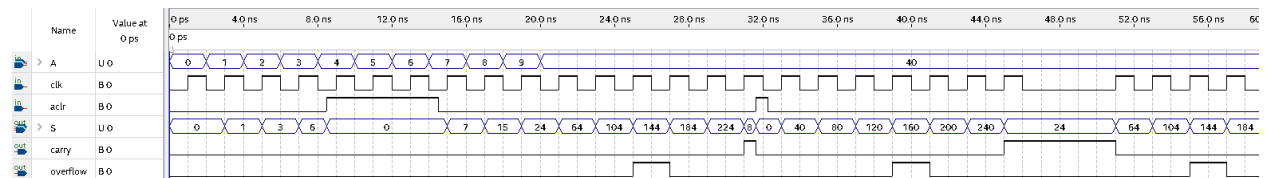
	Fmax	Restricted Fmax	Clock Name
1	429.92 MHz	429.92 MHz	clk

Z wyników syntezy Technology Map Viewer i kosztów zasobów sprzętowych wynika, że wersja 1 (akumulator z sumatorem z przeniesieniem szeregowym) używa 11 modułów logicznych (ALM) i 10 rejestrów. Natomiast wersja 2 (akumulator z operatorem przypisania ciągłego) używa 6 ALM i 10 rejestrów. Zatem koszt realizacji wersji 2 jest mniejszy niż koszt wersji 1.

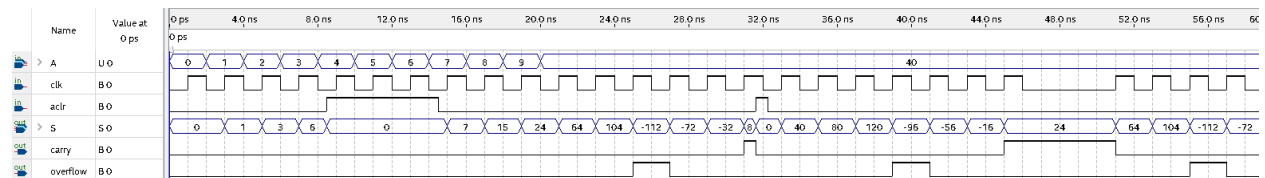
Wersja 1 jest w stanie pracować z sygnałem zegarowym clk o wyższej maksymalnej częstotliwości (547.95 MHz) niż wersja 2 (429.92 MHz). Zatem wersja 1 może być szybsza niż wersja 2.

Wyniki symulacji funkcjonalnej

Przeniesienie (carry = 1) występuje, jeżeli po dodaniu liczby A do sumy S akumulatora nowa wartość sumy S byłaby większa od 255 (2^8-1), czyli nie można byłoby jej zapisać jako 8-bitową liczbę bez znaku.



Przepełnienie (overflow = 1) występuje, jeżeli po dodaniu liczby A do sumy S akumulatora nowa wartość sumy S byłaby większa od 127 (2^7-1), czyli nie można byłoby jej zapisać jako 8-bitową liczbę ze znakiem.



Zadanie 2

Zmodyfikuj projekt z pierwszej części, aby można było dodawać i odejmować liczby. Aby to zrobić, dodaj wejście `add_sub` do projektu. Przy `add_sub = 0`, układ musi dodawać liczby do wartości sumy `S`, przy `add_sub = 1`, układ musi odjąć liczby od wartości sumy `S`. Przetestować układ. Sprawdź, czy sygnały **carry** i **overflow** są ustawiane podczas dodawania i odejmowania liczb.

Kod w Verilog

Moduł N-bitowego akumulatora

Tabela prawdy sygnału overflow

$S[N-1]$ – najbardziej znaczący bit aktualnej sumy akumulatora

$A[N-1]$ – najbardziej znaczący bit liczby na wejściu akumulatora

$S_plus_A[N-1]$ – najbardziej znaczący bit liczby na wyjściu sumatora/subtraktora

`add_sub` – 0, jeżeli dodajemy `A` do aktualnej sumy `S`; 1, jeżeli odejmujemy `A` od aktualnej sumy `S`

$S[N-1]$	$A[N-1]$	$S_plus_A[N-1]$	<code>add_sub</code>	overflow
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Wyznaczamy sygnał overflow metodą tabel Karnaugh.

$S[N-1] \ A[N-1] \setminus S_plus_A[N-1] \ add_sub$	00	01	11	10
00	0	0	0	1
01	0	0	0	0
11	1	0	0	0
10	0	1	0	0

overflow

```

overflow = (S[N-1] & A[N-1] & ~S_plus_A[N-1] & ~add_sub) |
(S[N-1] & ~A[N-1] & ~S_plus_A[N-1] & add_sub) |
(~S[N-1] & ~A[N-1] & S_plus_A[N-1] & ~add_sub)

```

Można zauważyć, że dla add_sub stałe równego 0 (tylko dodawanie) overflow upraszcza się do wzoru wyznaczonego w zadaniu 1.

```

overflow = (S[N-1] & A[N-1] & ~S_plus_A[N-1]) |
(~S[N-1] & ~A[N-1] & S_plus_A[N-1])

```

```

module accum_N_bits_add_sub
  #(parameter N = 8)(
    input [N-1:0] A,
    input clk, aclr, add_sub,
    output [N-1:0] S,
    output reg carry,
    output reg overflow);

  wire [N-1:0] S_plus_A;
  wire ad_cout;
  ripple_carry_adder_subtractor #(N) ad(S, A, add_sub, S_plus_A, ad_cout);
  register_N_bits #(N) r(S_plus_A, clk, aclr, S);

  wire of = (S[N-1] & A[N-1] & (~S_plus_A[N-1]) & (~add_sub)) |
    (S[N-1] & (~A[N-1]) & (~S_plus_A[N-1]) & add_sub) |
    ((~S[N-1]) & (~A[N-1]) & S_plus_A[N-1] & (~add_sub));
  always @ (posedge clk, posedge aclr)
    if (aclr) begin
      carry <= 0;
      overflow <= 0;
    end
    else begin
      carry <= add_sub ^ ad_cout;
      overflow <= of;
    end
endmodule

```

Moduł N-bitowego sumatora/subtraktora z przeniesieniem szeregowym

```

module ripple_carry_adder_subtractor
  #(parameter N = 4)(
    input [N-1:0] A, B,
    input sub,
    output [N-1:0] S,
    output cout);

  wire [N-2:0] c;
  generate
    genvar i;
    for (i = 0; i < N; i = i + 1)
      begin: ad
        case (i)
          0: full_adder fa(A[i], sub ^ B[i], sub, S[i], c[i]);
          N-1: full_adder fa(A[i], sub ^ B[i], c[i-1], S[i], cout);
          default: full_adder fa(A[i], sub ^ B[i], c[i-1], S[i], c[i]);
        endcase
      end
    endgenerate
endmodule

```

Moduł 1-bitowego pełnego sumatora

```
module full_adder(
    input a, b, cin,
    output s, cout);

    assign s = cin ^ (a ^ b);
    assign cout = a & b | (a ^ b) & cin;

endmodule
```

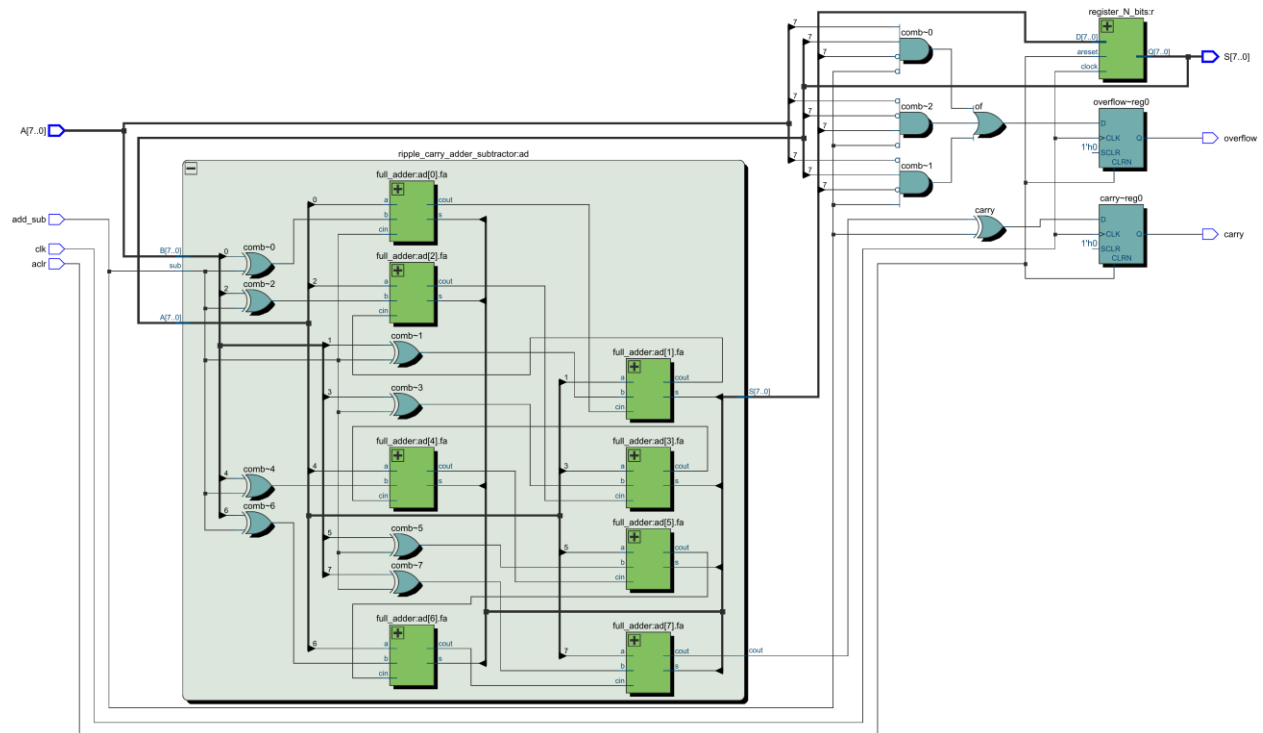
Moduł N-bitowego rejestru

```
module register_N_bits
    #(parameter N = 8)(
    input [N-1:0] D,
    input clock, areset,
    output reg [N-1:0] Q);

    initial Q = 0;
    always @ (posedge clock, posedge areset)
        if (areset) Q <= 0;
        else Q <= D;

endmodule
```

Wyniki syntezy

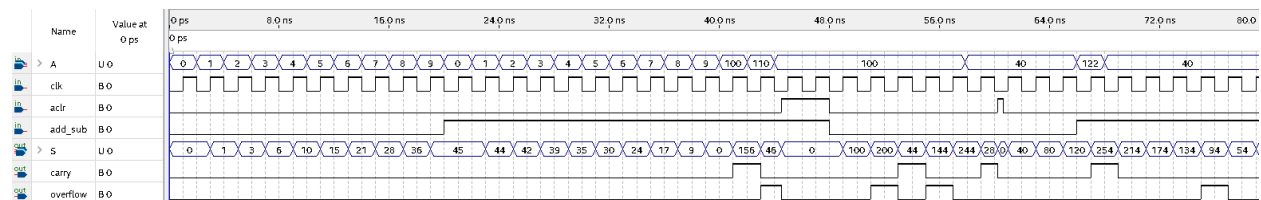


Wyniki symulacji funkcjonalnej

Sygnały carry i overflow są ustawiane podczas dodawania i odejmowania.

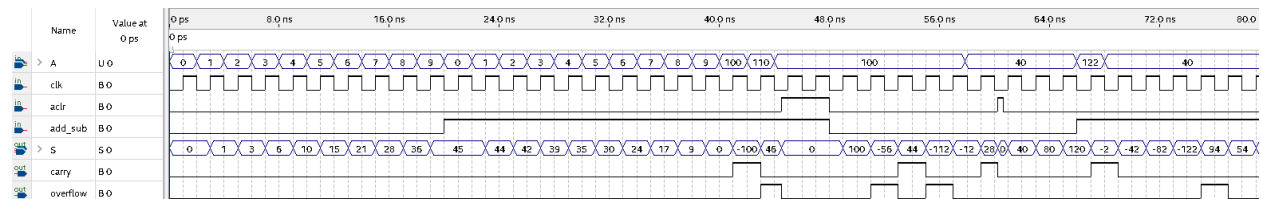
Przeniesienie (carry = 1) występuje, jeżeli po dodaniu liczby A do sumy S akumulatora nowa wartość sumy S byłaby większa od 255 (2^8-1), czyli nie można byłoby jej zapisać jako 8-bitową liczbę bez znaku.

Jeżeli po odjęciu liczby A od sumy S akumulatora nowa wartość sumy S byłaby mniejsza od 0, to również carry = 1 i interpretujemy to jako pożyczkę.



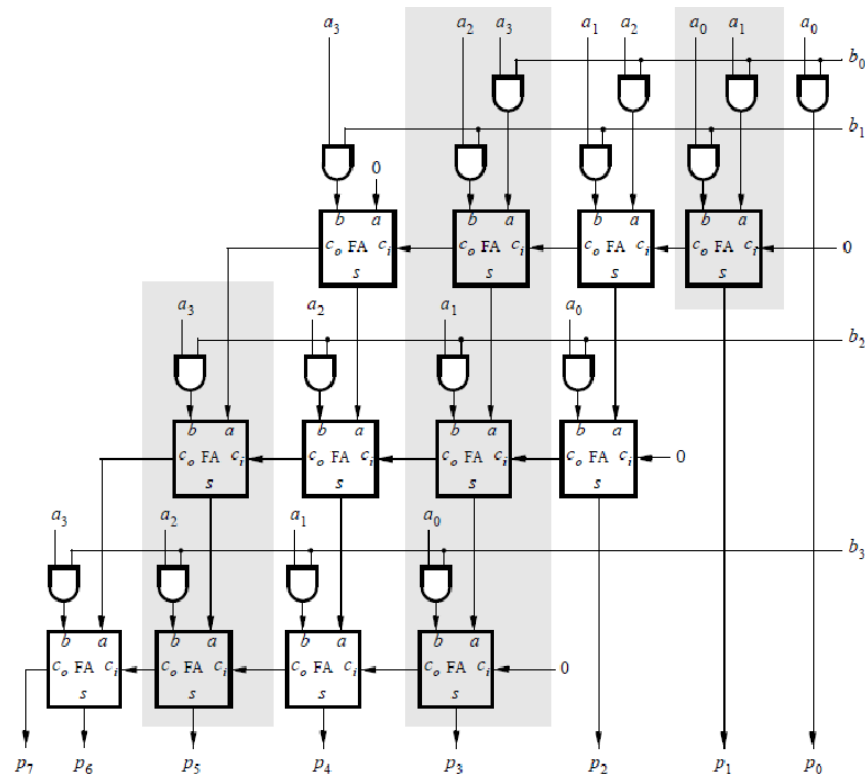
Przepełnienie (overflow = 1) występuje, jeżeli po dodaniu liczby A do sumy S akumulatora nowa wartość sumy S byłaby większa od 127 (2^7-1), czyli nie można byłoby jej zapisać jako 8-bitową liczbę ze znakiem.

Jeżeli po odjęciu liczby A od sumy S akumulatora nowa wartość sumy S byłaby mniejsza od -128 (-2^7), to również występuje przepełnienie i overflow = 1.



Zadanie 3

Utwórz projekt array_multiplier_4_bits mnożnika macierzowego 4-bitowego o strukturze jak na rys 3.



Rys. 3. Schemat mnożnika macierzowego

z następującymi wyprowadzeniami:

Wyprowadzenia	Piny
A[3:0]	SW7-4, HEX2
B[3:0]	SW3-0, HEX0
P[7:0]	HEX5-4

Wykonaj symulację funkcjonalną obwodu, a następnie przetestuj ją na płycie.

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module array_multiplier_4_bits_on_board(  
    input [7:0] Sw,  
    output [6:0] HEX2, HEX0, HEX5, HEX4);  
  
    wire [3:0] A = Sw[7:4], B = Sw[3:0];  
    wire [7:0] P;  
    array_multiplier_4_bits mul(A, B, P);  
  
    decoder_hex_16(A, HEX2);  
    decoder_hex_16(B, HEX0);  
    decoder_hex_16(P[7:4], HEX5);  
    decoder_hex_16(P[3:0], HEX4);  
  
endmodule
```

Moduł dekodera z kodu binarnego na kod wyświetlacza 7-segmentowego

```
module decoder_hex_16(  
    input [3:0] binary,  
    output reg [6:0] h);  
  
    always @ (*)  
        case (binary)  
            4'd0: h = 7'b1000000; // 64  
            4'd1: h = 7'b1111001; // 121  
            4'd2: h = 7'b0100100; // 36  
            4'd3: h = 7'b0110000; // 48  
            4'd4: h = 7'b0011001; // 25  
            4'd5: h = 7'b0010010; // 18  
            4'd6: h = 7'b0000010; // 2  
            4'd7: h = 7'b1111000; // 120  
            4'd8: h = 7'b0000000; // 0  
            4'd9: h = 7'b0010000; // 16  
            4'd10: h = 7'b0001000; // 8; A  
            4'd11: h = 7'b0000011; // 3; b  
            4'd12: h = 7'b0100111; // 39; c  
            4'd13: h = 7'b0100001; // 33; d  
            4'd14: h = 7'b0000110; // 6; E  
            4'd15: h = 7'b0001110; // 14; F  
        endcase  
  
endmodule
```

Moduł 1-bitowego pełnego sumatora

```
module full_adder(  
    input a, b, cin,  
    output s, cout);  
  
    assign s = cin ^ (a ^ b);  
    assign cout = a & b | (a ^ b) & cin;  
  
endmodule
```

Moduł 4-bitowego macierzowego mnożnika z zachowaniem przeniesienia (Carry-Save Array Multiplier)

```

module array_multiplier_4_bits(
    input [3:0] A, B,
    output [7:0] P);

    wire ab[3:0][3:0]; // 1-bitowe iloczyny a*b
    generate
        genvar ai, bi;
        for (ai = 0; ai <= 3; ai = ai + 1) begin: fora
            for (bi = 0; bi <= 3; bi = bi + 1) begin: forb
                assign ab[ai][bi] = A[ai] & B[bi];
            end
        end
    endgenerate

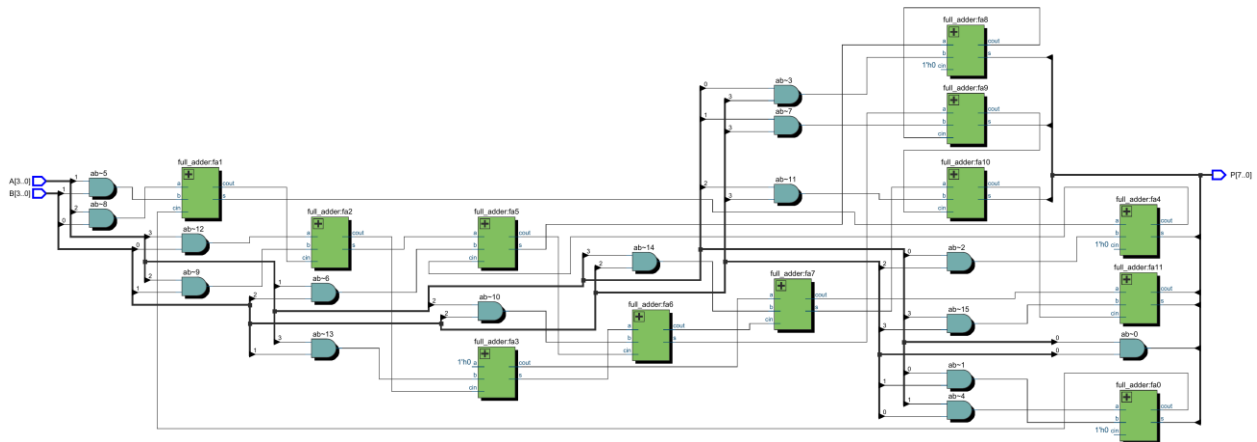
    wire s[11:0], cout[11:0]; // 1-bitowe sumy i przeniesienia z pełnych sumatorów
    full_adder fa0(ab[1][0], ab[0][1], 1'b0, s[0], cout[0]);
    full_adder fa1(ab[2][0], ab[1][1], cout[0], s[1], cout[1]);
    full_adder fa2(ab[3][0], ab[2][1], cout[1], s[2], cout[2]);
    full_adder fa3(1'b0, ab[3][1], cout[2], s[3], cout[3]);
    full_adder fa4(s[1], ab[0][2], 1'b0, s[4], cout[4]);
    full_adder fa5(s[2], ab[1][2], cout[4], s[5], cout[5]);
    full_adder fa6(s[3], ab[2][2], cout[5], s[6], cout[6]);
    full_adder fa7(cout[3], ab[3][2], cout[6], s[7], cout[7]);
    full_adder fa8(s[5], ab[0][3], 1'b0, s[8], cout[8]);
    full_adder fa9(s[6], ab[1][3], cout[8], s[9], cout[9]);
    full_adder fa10(s[7], ab[2][3], cout[9], s[10], cout[10]);
    full_adder fa11(cout[7], ab[3][3], cout[10], s[11], cout[11]);

    assign P = { cout[11], s[11], s[10], s[9], s[8], s[4], s[0], ab[0][0] };

endmodule

```

Wyniki syntezy

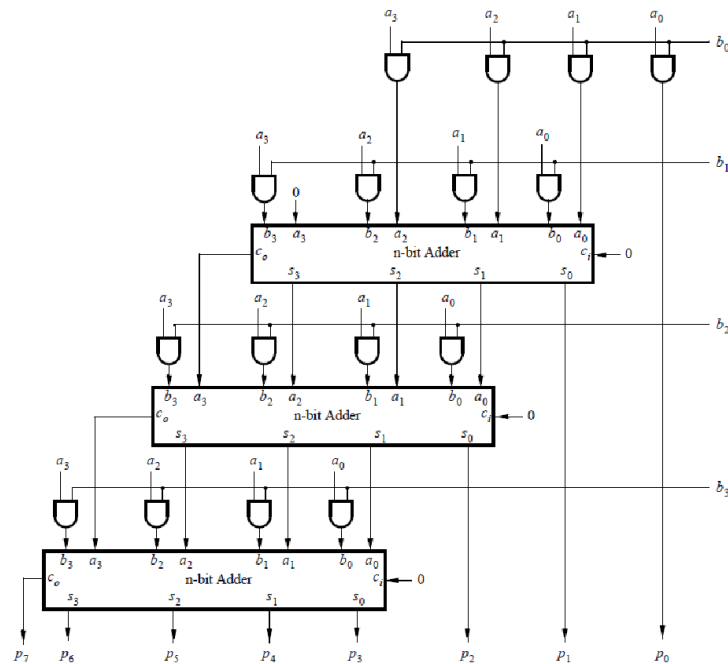


Wyniki symulacji funkcjonalnej

Name	Value at 0 ps	0 ps	10.0 ns	20.0 ns	30.0 ns	40.0 ns	50.0 ns	60.0 ns	70.0 ns	80.0 ns
> A	U 15	15	6	0	3	12	6	10	15	
> B	U 6	6	0	4	11	10	7	3	15	
> P	U 90	90	0		33	120	42	30	225	

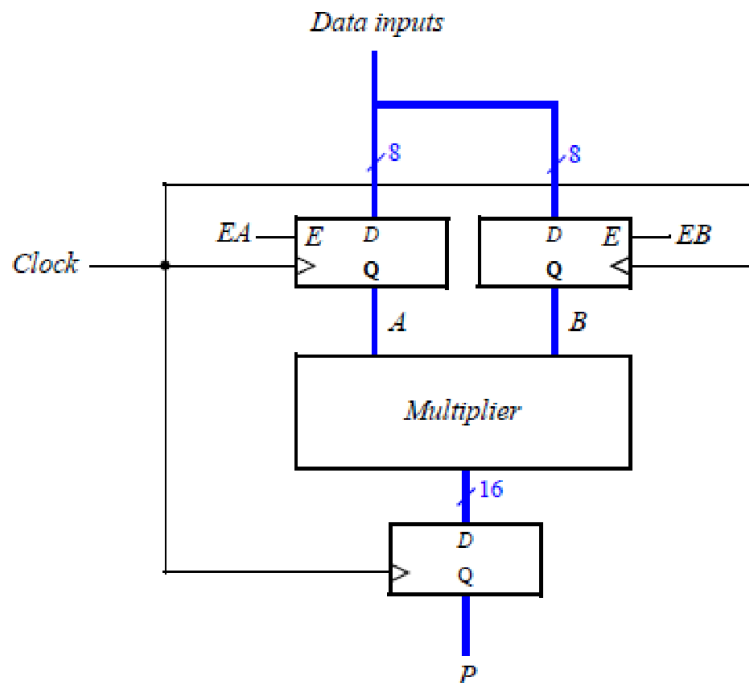
Zadanie 4

Utwórz projekt multiplier_N_bits mnożnika N-bitowego o strukturze jak na rys. 4.



Rys. 4. Realizacja mnożnika macierzowego za pomocą n-bitowych sumatorów

Tutaj, sumatory N-bitowe są używane do tworzenia częściowych produktów. Wynik jest tworzony przez dodanie przesuniętych wartości częściowych produktów. Zastosuj 8-bitowy układ mnożnika N-bitowego o strukturze jak na rys. 5.



Rys. 5. Schemat mnożnika z buforem

z następującymi wyprowadzeniami:

Wyprowadzenia	Piny
Data[7:0]	SW7-0
clk	KEY1
aclr	KEY0
EA	SW9
EB	SW8
P[15:0]	HEX3-0
A	LEDR[7:0] przy SW9=1
B	LEDR[7:0] przy SW8=1

Wykonaj symulację funkcjonalną obwodu, a następnie przetestuj ją na płycie.

Wskazówka: Aby zrealizować projekt, utwórz następujące moduły:

register_N_bits_ena_aclr - rejestr dla N bitów z włączoną synchronizacją (ena) i asynchronicznym resetem (aclr)

adder_N_bits - sumator dla N bitów z wejściem przeniesienia (cin) i wyjściem przeniesienia (cout)

multiplier_N_bits # (parametr N = 8) (input [N-1: 0] A, B, output [2 * N: 0] P); - Mnożnik N-bitowy, który implementuje strukturę z rys. 4.

W kodzie mnożnika użyj bloków generujących (generate), aby wygenerować częściowe produkty, częściowe sumy i wynik.

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module buffered_multiplier_N_bits_on_board(  
    input [9:0] Sw,  
    input [1:0] KEY,  
    output [6:0] HEX3, HEX2, HEX1, HEX0,  
    output [7:0] LEDR);  
  
    wire [15:0] P;  
    buffered_multiplier_N_bits #(8) buf_mul(SW[7:0], KEY[1], KEY[0], SW[9], SW[8], P, LEDR);  
  
    decoder_hex_16 dec3(P[15:12], HEX3);  
    decoder_hex_16 dec2(P[11:8], HEX2);  
    decoder_hex_16 dec1(P[7:4], HEX1);  
    decoder_hex_16 dec0(P[3:0], HEX0);  
  
endmodule
```

Moduł dekodera z kodu binarnego na kod wyświetlacza 7-segmentowego

```
module decoder_hex_16(  
    input [3:0] binary,  
    output reg [6:0] h);  
  
    always @ (*)  
        case (binary)  
            4'd0: h = 7'b1000000; // 64  
            4'd1: h = 7'b1111001; // 121  
            4'd2: h = 7'b0100100; // 36  
            4'd3: h = 7'b0110000; // 48  
            4'd4: h = 7'b0011001; // 25  
            4'd5: h = 7'b0010010; // 18  
            4'd6: h = 7'b0000010; // 2  
            4'd7: h = 7'b1111000; // 120  
            4'd8: h = 7'b0000000; // 0  
            4'd9: h = 7'b0010000; // 16  
            4'd10: h = 7'b0001000; // 8; A  
            4'd11: h = 7'b0000011; // 3; b  
            4'd12: h = 7'b0100111; // 39; c  
            4'd13: h = 7'b0100001; // 33; d  
            4'd14: h = 7'b0000110; // 6; E  
            4'd15: h = 7'b0001110; // 14; F  
        endcase  
  
endmodule
```

Moduł mnożnika z buforami (rejestrami)

```
module buffered_multiplier_N_bits  
    #(parameter N = 8)(  
        input [N-1:0] Data,  
        input clk, aclr, EA, EB,  
        output [2*N-1:0] P,  
        output [N-1:0] A_B);  
  
    wire [N-1:0] A, B;  
    register_N_bits_ena_aclr #(N) r_A(Data, clk, aclr, EA, A);  
    register_N_bits_ena_aclr #(N) r_B(Data, clk, aclr, EB, B);  
    mux_4_1 #(N) mux({N{1'b0}}, B, A, {N{1'b0}}, {EA, EB}, A_B);  
  
    wire [2*N-1:0] mul_P;  
    multiplier_N_bits #(N) mul(A, B, mul_P);  
    register_N_bits_ena_aclr #(2*N) r_P(mul_P, clk, aclr, 1'b1, P);  
  
endmodule
```

Moduł N-bitowego rejestru

```
module register_N_bits_ena_aclr  
    #(parameter N = 8)(  
        input [N-1:0] D,  
        input clock, aclr, ena,  
        output reg [N-1:0] Q);  
  
    initial Q = 0;  
    always @ (posedge clock, posedge aclr)  
        if (aclr) Q <= 0;  
        else if (ena) Q <= D;  
        else Q <= Q;  
  
endmodule
```

Moduł N-bitowego multiplexera 4x1

```
module mux_4_1
  #(parameter N = 8)(
    input [N-1:0] inf0, inf1, inf2, inf3,
    input [1:0] s,
    output reg [N-1:0] out);

  always @ (*)
    case (s)
      2'd0: out = inf0;
      2'd1: out = inf1;
      2'd2: out = inf2;
      2'd3: out = inf3;
    endcase

endmodule
```

Moduł N-bitowego mnożnika macierzowego używającego N-bitowych sumatorów

```
module multiplier_N_bits
  #(parameter N = 4)(
    input [N-1:0] A, B,
    output [2*N-1:0] P);

  wire [N-1:0] pp[N-1:0]; // częściowe iloczyny
  wire [N-1:0] s[N-1:1]; // częściowe sumy
  wire cout[N-1:1]; // przeniesienia
  genvar i;
  // tworzenie częściowych iloczynów
  generate
    for(i = 0; i <= N - 1; i = i + 1) begin: b11
      assign pp[i] = A & {N{B[i]}};
    end
  endgenerate
  // tworzenie częściowych sum
  adder_N_bits #(N) ad1({1'b0, pp[0][N-1:1]}, pp[1], 1'b0, s[1], cout[1]);
  generate
    for(i = 2; i <= N - 1; i = i + 1) begin: b12
      adder_N_bits #(N) adi({cout[i-1], s[i-1][N-1:1]}, pp[i], 1'b0, s[i], cout[i]);
    end
  endgenerate
  // tworzenie ostatecznego iloczynu
  assign P[0] = pp[0][0];
  generate
    for(i = 1; i <= N - 2; i = i + 1) begin: b13
      assign P[i] = s[i][0];
    end
  endgenerate
  assign P[2*N-2:N-1] = s[N-1];
  assign P[2*N-1] = cout[N-1];

endmodule
```

Moduł N-bitowego sumatora z przeniesieniem szeregowym (Ripple-Carry Adder)

```

module adder_N_bits
  #(parameter N = 4)(
    input [N-1:0] A, B,
    input cin,
    output [N-1:0] S,
    output cout);

  wire [N-2:0] c;
  generate
    genvar i;
    for (i = 0; i < N; i = i + 1)
      begin: ad
        case (i)
          0: full_adder fa(A[i], B[i], cin, S[i], c[i]);
          N-1: full_adder fa(A[i], B[i], c[i-1], S[i], cout);
          default: full_adder fa(A[i], B[i], c[i-1], S[i], c[i]);
        endcase
      end
    endgenerate
endmodule

```

Moduł 1-bitowego pełnego sumatora

```

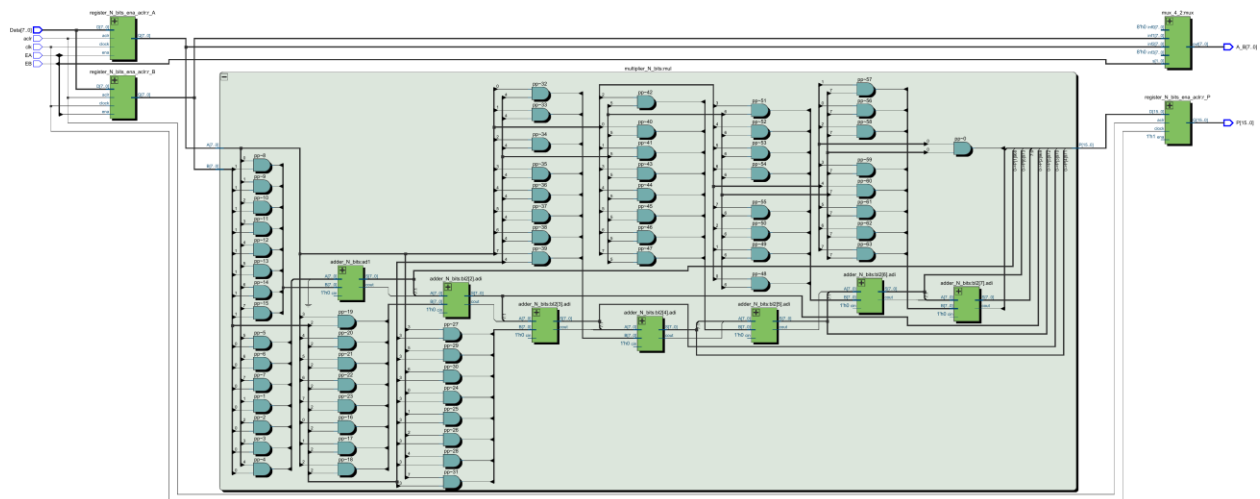
module full_adder(
  input a, b, cin,
  output s, cout);

  assign s = cin ^ (a ^ b);
  assign cout = a & b | (a ^ b) & cin;

endmodule

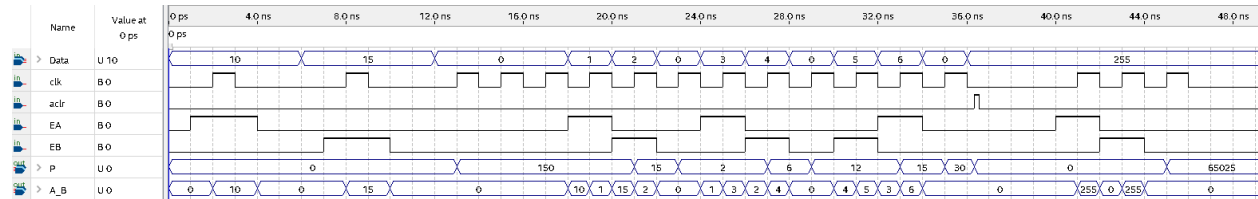
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej

W dwóch taktach zegara clk za pomocą 1 na wejściach EA i EB wprowadzamy do rejestrów A i B operandy mnożenia z wejścia Data. Następnie w trzecim takcie clk zapamiętane operandy są mnożone, a wynik jest wprowadzany do rejestru P. Na wyjście A_B przekazywany jest operand zapisany w rejestrze A lub B odpowiednio przy EA = 1 lub EB = 1. Przy EA = EB wyjście A_B = 0.



Wnioski

Wszystkie zadania zostały zrealizowane.